Accelerated array computing and code transformations for R

Sebastian Fischer[1,2] · Daniel Falbel · Tomasz Kalinowski[3] · Niko German · Martin Binder[1,2] · Bernd Bischl[1,2]

[1]LMU Munich [2]Munich Center for Machine Learning (MCML) [3]Posit

Motivation

anvl brings **compiled, GPU-accelerated array computing** to R: you write numerical programs in plain R, which anvl compiles to optimized machine code via **OpenXLA** — the compiler stack underpinning frameworks like TensorFlow, JAX or PyTorch (TPU). anvl aspires to be for R what JAX is for Python. Unlike eager-first frameworks such as **torch**, anvl focuses on compilation, which means it requires fewer hand-written CUDA kernels without sacrificing performance.

Create `AnvlArray`s from R objects with `nv_array()` — the `device` argument picks the hardware. Default `dtype` is `f32` and device `"cpu"`.

```
X     <- nv_array(rnorm(N * P), shape = c(N, P), dtype = "f32")
beta  <- nv_array(rnorm(P), shape = c(P, 1), device = "cpu")
alpha <- nv_scalar(rnorm(1))
y     <- nv_array(rnorm(N), shape = c(N, 1))
```

① Eager mode

Each operation (`%*%`, `+`) is **dispatched and run immediately**. Primarily intended for interactive use and debugging.

```
linear <- function(X, beta, alpha)
  X %*% beta + alpha
y_hat <- linear(X, beta, alpha)
```

② JIT mode

`jit()` **compiles the whole function as a single unit**, allowing for joint optimizations like kernel fusion or buffer reuse. Shapes and data types are specialized and subsequent calls with the same input shapes and dtypes use the cached executable.

```
fast  <- jit(linear)
y_hat <- fast(X, beta, alpha)
```

③ Automatic differentiation

`gradient()` returns a new function that computes **reverse-mode gradients** of an R function. This is currently the only example of a semantic code transformation.

```
loss <- function(X, y, beta, alpha)
  mean((linear(X, beta, alpha) - y)^2)
grad <- jit(gradient(loss, wrt = "beta"))
g    <- grad(X, y, beta, alpha)
```

Why anvl?

- same code runs on CPU and GPU
- a more consistent array API than base R
- simplified CUDA install (only an NVIDIA driver needed)
- JIT compilation sets it apart from **torch**
- written primarily in R, so easy to extend and contribute to
- around 140 common functions already covered and more coming
- possible use-cases: gradient-based methods, Monte Carlo samplers, expensive matrix algebra, etc.
- separates the **what** from the **how**: an alternative, experimental **quickr** backend transpiles to Fortran instead.

✓ Fast when

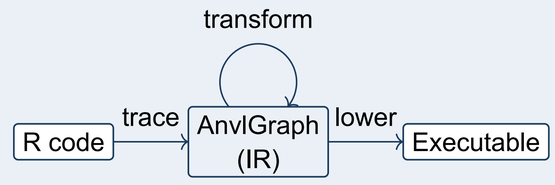
- The compilation overhead is amortized, either because:
  - the function is called repeatedly with the same shapes and data types (can be enforced via padding)
  - a single function call is substantial
- The work involves dense arrays

✗ Not great (yet) when

- Data-dependent shape computations cannot be part of a compiled function
- The data is sparse

How it works

anvl pipes your code through three types of **code transformations**, shown below on the tiny example `f <- function(x) x * 2` — each transformation called directly, with its output.



① Trace — R → AnvlGraph

Run the function on abstract inputs and record primitive operations.

```
trace_fn(f, args = list(
  x = nv_aval("f32", integer())))
```

```
<AnvlGraph>
  Inputs:
    %x1: f32[]
  Body:
    %1: f32[] = mul(%x1, 2:f32?)
  Outputs:
    %1: f32[]
```

② Transform — AnvlGraph → AnvlGraph

Rewrite the graph, e.g. to compute its gradient.

```
transform_gradient(graph, wrt = "x")
```

```
<AnvlGraph>
  Inputs:
    %x1: f32[]
  Constants:
    %c1: f32[]
  Body:
    %1 = mul(%x1, 2:f32?)
    %2 = mul(%c1, 2:f32?)
  Outputs:
    %2: f32[]
```

③ Lower — AnvlGraph → Executable

Compile the graph into a hardware-specific executable. Built on the **pjrt** and **stablehlo** packages, which are developed in tandem.

```
hlo <- stablehlo(graph)[[1]]
exe <- pjrt_compile(pjrt_program(repr(hlo)))
print(hlo)
```

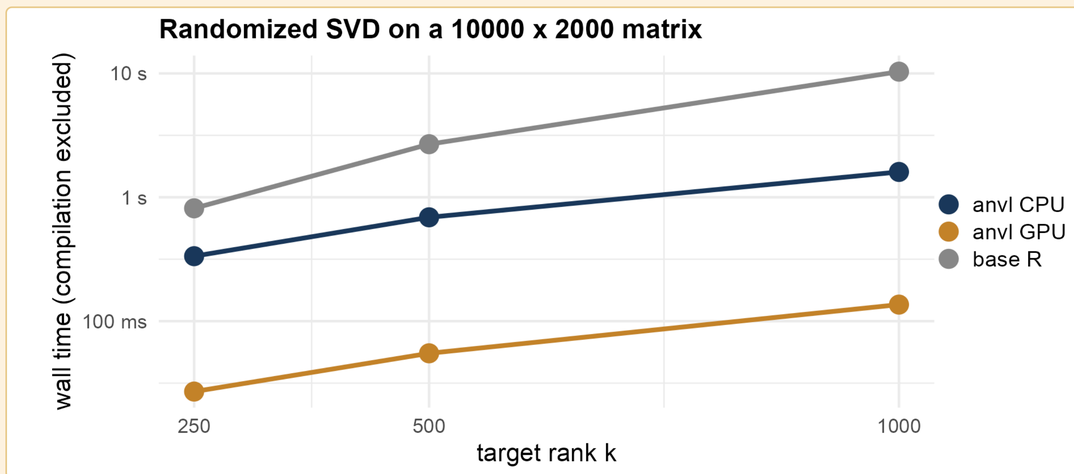
```
func.func @main(%0: tensor<f32>) -> tensor<f32> {
  %1 = stablehlo.constant dense<2.0e0> : tensor<f32>
  %2 = stablehlo.multiply %0, %1 : tensor<f32>
  return %2 : tensor<f32>
}
```

Application: randomized SVD

For $A \in \mathbb{R}^{n \times p}$ with n, p large but only the top $k \ll \min(n, p)$ singular triplets needed, the randomized SVD (Halko et al., 2011) approximates them in four steps: sample $\Omega \in \mathbb{R}^{p \times k} \sim \mathcal{N}(0, 1)$; sketch $Y = A\Omega \in \mathbb{R}^{n \times k}$; orthonormalize $QR = Y$; and SVD the small matrix $B = Q^T A = \tilde{U} \Sigma V^T$. Then $A \approx Q\tilde{U}\Sigma V^T$, giving the approximate top-k triplets $\left(\Sigma, Q\tilde{U}, V^T\right)$.

```
k <- 200L
rsvd <- jit(function(A, key, k) {
  Omega <- nv_rnorm(c(ncol(A), k), key)[[2]]
  Q <- nv_qr(A %*% Omega)$Q
  B <- t(Q) %*% A
  s <- nv_svd(B)
  list(d = s$d, u = Q %*% s$u, vt = s$vt)
}, static = "k")
```

Based on Gundersen (2019), gregorygundersen.com/blog/2019/01/17/randomized-svd.



Supported by **MaRDI** — the Mathematical Research Data Initiative, an NFDI project funded by DFG grant 460135501.